ROAD RISK ANALYSIS PLATFORM

# Functional & API Specification

A consolidated reference of the platform's functional behaviour and REST API contracts, for the team building the VIASafe mobile application.

Prepared for: third-party mobile development team

Scope: Dashboard, Detection & Clustering, Live Map, Job & Upload Management, Validations, Community Reporting, Driver Behaviour Analytics, Predictive Collision Prevention

## Contents


1. Dashboard
2. Detection Location & Path Handling
3. Cluster Configuration
4. Map Provider & Access Rights
5. Live Map
6. Hazard Track Page
7. Jobs
8. Upload Video
9. Validations
10. Community Reporting Module
11. Driver Behaviour Analytics Module
12. Predictive Cooperative Collision Prevention Module

# 1. Dashboard

The dashboard is the administrator's landing view onto platform activity. It is a presentation layer only — it consumes existing data from the modules described below and applies no independent business logic of its own. There is no dedicated dashboard API beyond the endpoints already listed under each module.

## 2. Detection Location & Path Handling

Detections are never assigned to a predefined or manually-selected path. Each detection's position comes directly from the GPS telemetry embedded in the source video at the moment the hazard is captured. Once a coordinate pair is available, the platform automatically resolves it to a human-readable location (road, locality, or administrative zone) through reverse geocoding, and this resolved location travels with the detection through clustering, review, and display.

Clusters inherit their location the same way: a cluster's position and label are derived automatically from the detections that compose it, never from an operator-assigned route.

## 3. Cluster Configuration

Detections are grouped into clusters using a single, platform-wide clustering radius — not a per-road or per-highway value. Administrators manage this radius from Settings.

### ENDPOINTS

**GET** `/cluster-config/`

Returns the current clustering configuration.

```
→ { "radius_m": 50.0 }
```

**PATCH** `/cluster-config/`

Updates the clustering radius.

```
{ "radius_m": 75.0 }
```

A future revision of this configuration may separate the radius into an Urban (In Agglomeration) and a Rural (Out of Agglomeration) value, each applied depending on the caller's environment; the mobile client should not assume a single global radius will remain the only supported mode.

## 4. Map Provider & Access Rights

The platform renders maps using open-source providers throughout: MapLibre GL with vector tiles on mobile, and Leaflet with OpenStreetMap tiles on the web admin interface. There is no provider-selection setting and no Free/Premium application mode — every environment uses the same open-source mapping stack, and no mapping API key is required.

## 5. **Live Map**

Routes are computed by OSRM and rendered using the full route geometry returned by the routing engine — the drawn path follows the actual road network, including curves and junctions, not a straight line between origin and destination.

Along any computed route, the client can request all hazard clusters located within a configurable corridor around the route, so the map can highlight relevant detections ahead on the journey.

### ENDPOINT

**POST** `/detections/along-route`

```
{ "waypoints": [ {"latitude": 4.05, "longitude": 9.71}, ... ], "corridor_m": 60 }
→ { "total": 12, "items": [ Detection, ... ] }
```

waypoints should be the full decoded route geometry (every bend), not just origin and destination, so hazards on curved sections are not missed.

## 6. **Hazard Track Page**

Detected hazards are grouped into geographic clusters automatically and continuously — no manual path creation step is required at any point in this flow.

### ENDPOINT

**POST** `/detections/clusters`

```
{ "bounds": {...}, "zoom": 14 }
→ { "clusters": [ { "id", "latitude", "longitude", "member_count", "detections": [...]
} ] }
```

Returns clusters sized for the current map viewport; each detection within a cluster carries its own resolved location name.

# 7. Jobs

The Jobs page is a monitoring surface for video-processing tasks: it tracks progress and status only, and does not host review actions or the upload flow — those live in their own dedicated areas (see Validations and Upload Video).

### ENDPOINTS

**GET** `/jobs/`

Lists jobs with their processing status.

**GET** `/jobs/{id}`

Returns a single job's detail, including processed/total frame counts and its resulting detections.

## 8. Upload Video

Uploading a video requires no manual vehicle metadata and no path assignment — every location is derived later from the video's own embedded GPS telemetry. The upload form supports selecting multiple files at once, and shows the AI model(s) that will process each one.

### ENDPOINT

**POST** `/jobs/?models=pothole,signs,speedbump`

```
multipart/form-data: file
→ 202 Accepted, { "id", "status": "pending", ... }
```

One call per video file; the queued job is then tracked from the Jobs page.

Jobs are queued server-side and started as processing capacity becomes available. A future revision may also accept a video by reference — for example a signed URL to a cloud storage location (Drive, OneDrive, a shared repository) — in addition to a direct file upload, via a dedicated endpoint such as `POST /jobs/from-url`.

# 9. **Validations**

Each detection carries a resolved, human-readable location (see Section 2) alongside its raw coordinates, and a review status: pending, validated, or rejected. Reviewing a detection never deletes it — rejecting a detection sets its status to rejected and keeps the full record, including its image and metadata, available for later reference.

### ENDPOINTS

**PATCH** `/detections/{id}/review`

```
{ "status": "validated" | "rejected", "label"?, "severity_score"?, "note"? }
```

Records the reviewer's decision. Validating a detection also stores a labeled training example; a future revision extends this to rejected detections as well, so both outcomes feed the training dataset.

**PATCH** `/detections/{id}/location`

```
{ "latitude": 4.06, "longitude": 9.73 }
```

Lets a reviewer correct a detection's location manually — used when the video's embedded GPS was noisy, partial, or momentarily missing. On save, the corrected coordinates are automatically re-resolved to a location name.

Detections whose location could not be automatically determined are still created and retained for manual correction (rather than discarded), together with the surrounding video context (roughly 3 seconds before and after the detected event) so a reviewer can identify and fix the position.

# 10. Community Reporting Module

End users can report road events directly from the mobile application, contributing a second source of information alongside AI-based hazard detection. A report carries a type, a GPS position, and optionally a description, a photo, and a voice recording.

## Report Types

Traffic congestion, police presence, road accidents, road hazards, road closures, blocked lanes, map issues, adverse weather conditions, roadside assistance requests, and places of interest.

## Community Validation

Reports are validated by the community rather than by an administrator: other users can confirm that a reported event is still present, or indicate that it is no longer there. A report that accumulates enough "not there" votes is automatically retired.

## Report Visibility

Nearby and along-route queries take a search radius / corridor width supplied by the caller. A future revision centralises these as platform Settings, with separate values for Urban (In Town) and Rural (Out of Town) environments.

### CREATE A REPORT

**POST** `/signalements/`

```
multipart/form-data:
  type, latitude, longitude,
  description?, image?, audio?
→ 201, Signalement
```

### RETRIEVE NEARBY REPORTS

**POST** `/signalements/nearby`

```
{ "latitude": 4.05, "longitude": 9.71, "radius_m": 5000 }
→ { "total": n, "items": [ Signalement, ... ] }
```

### RETRIEVE REPORTS ALONG A ROUTE

**POST** `/signalements/along-route`

```
{ "waypoints": [...], "corridor_m": 60 }
→ { "total": n, "items": [ Signalement, ... ] }
```

### CONFIRM / REJECT A REPORT

**POST** `/signalements/{id}/confirm`

Increments the report's confirmation count.

**POST** `/signalements/{id}/not-there`

Increments the "not there" count; the report is automatically retired once this reaches the configured threshold (currently 3).

A future revision adds the caller's own position to both calls, so a vote is only accepted when the user is within the configured validation radius of the reported event, and extends confirmations to influence a confidence score and validity window on the report itself.

### AI-DETECTED ROAD HAZARDS (NEARBY / ALONG ROUTE)

**POST** `/detections/nearby`

```
{ "latitude", "longitude", "radius_m" }
→ { "total": n, "items": [ Detection, ... ] }
```

**POST** `/detections/along-route`

```
{ "waypoints": [...], "corridor_m" }
→ { "total": n, "items": [ Detection, ... ] }
```

## Configuration

| Setting | Current form |
|---|---|
| Report visibility radius | Per-request parameter (radius_m on /nearby) |
| Report validation radius | Not yet a distinct setting — planned |
| AI detection search radius | Per-request parameter (radius_m on /detections/nearby) |
| Route corridor width | Per-request parameter (corridor_m) |
| Cluster radius — Urban / Rural | Single global value today (see Section 3); Urban/Rural split planned |

# 11. Driver Behaviour Analytics Module

Connected vehicles transmit telemetry — position, speed, heading, and acceleration — while in motion. This module classifies that telemetry into discrete driving-behaviour events and aggregates them geographically to surface high-risk zones.

## Event Types

Speeding, harsh braking, sudden acceleration, harsh cornering, rapid deceleration, driver stop duration, average speed by segment, and traffic flow indicators.

## Risk Zone Identification

Events are aggregated by geographic cell with a weighted severity score, so that locations with a disproportionate concentration of harsh-braking, speeding, or accident-related events surface automatically as high-risk zones, without manual tagging.

### PROPOSED API

**POST** `/driving-events`

```
{ "vehicle_id", "event_type", "latitude", "longitude",
  "speed_mps", "timestamp" }
→ 201
```

Submits a single live driving-behaviour event from the mobile app.

**GET** `/historical/risk/zone`

Returns the aggregated risk score for a geographic cell.

**POST** `/historical/risk/route`

Returns risk-weighted cells along a planned route.

**GET** `/driving-events/stats`

Returns aggregated statistics — average speed by segment, traffic-flow indicators, event counts by type and zone.

## Configurable Thresholds

Speed, braking-intensity, and acceleration thresholds, along with the event-aggregation radius, are configurable per road category and per Urban/Rural environment via Settings.

# 12. Predictive Cooperative Collision Prevention Module

Every connected vehicle running the mobile application periodically transmits its position, speed, heading, timestamp, vehicle identifier, and vehicle type to the platform. A central component processes this telemetry, maintains each vehicle's predicted trajectory, evaluates the risk of the vehicle interacting with others nearby, and delivers an alert to the affected vehicles when that risk crosses a configured threshold.

### TELEMETRY

**POST** `/vehicles/register`

```
{ "vehicle_id", "vehicle_type" }
→ Vehicle
```

**POST** `/vehicles/{id}/ping`

```
{ "latitude", "longitude", "speed_mps", "heading",
  "accuracy_m"?, "acceleration_mps2"?, "ts"? }
→ 204 No Content
```

Called periodically while the vehicle is in motion. Triggers the platform's road-state, trajectory, risk, and alert pipeline for this vehicle.

## Road-Aware Filtering

Rather than comparing every vehicle to every other vehicle by raw distance, the platform narrows candidates using the current road segment, direction of travel, and predicted trajectory, so that two vehicles a few metres apart on unrelated roads do not trigger an alert.

## Risk Evaluation

For each pair of vehicles that may realistically interact within the prediction horizon, the platform computes distance, relative speed, and time-to-collision, and classifies the situation into an alert level (for example: Monitor, Caution, Warning, Critical). Only levels above a configured threshold generate a driver notification.

### ALERTS & NEARBY VEHICLES

**GET** `/alerts/{vehicle_id}`

Returns the vehicle's current alert, if any, with level, distance, and time-to-collision.

**GET** `/alerts/active`

Returns all currently active alerts platform-wide.

**POST** `/vehicles/nearby`

```
{ "latitude", "longitude", "radius_m", "exclude_vehicle_id"? }
→ { "total", "items": [ VehicleLive, ... ] }
```

**GET** `/vehicles/live`

Returns every currently active vehicle, unfiltered by position.

### Driver Notification

Notifications currently reach the driver through the mobile app's own UI: a screen banner, a map overlay, and device vibration. A future revision may add voice guidance and an audible alarm as additional channels.

### Road Context

Vehicle positions are matched to the underlying open-source road network (see Section 4), which lets the prediction engine distinguish, for example, two vehicles travelling in opposite directions on the same road from two vehicles on unrelated, nearby roads.

### Communication Model

1. Each connected vehicle periodically transmits its position and movement data.
2. The platform ingests the telemetry and updates the vehicle's road-aware state.
3. The prediction engine updates the vehicle's projected trajectory.
4. The risk engine evaluates potential conflicts with nearby vehicles.
5. Only vehicles concerned by a predicted risk receive an alert.
6. The driver takes the appropriate action from within the mobile app.

The mobile application requires an active network connection for both map tiles and telemetry; offline map caching and deferred synchronisation are not part of the current contract.